



10-x23 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

Variational Autoencoders (VAEs)

Matt Gormley
Lecture 9

Reminders

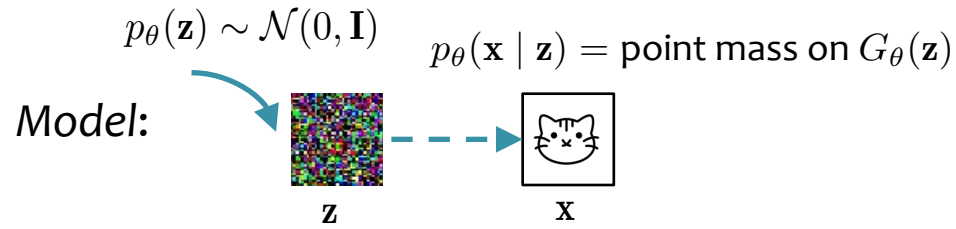
- **Quiz 2:**
 - In-class: Mon, Feb 16
 - Lectures 5-9 (only the VAE part of Lecture 9)
- **Homework 2: Generative Models of Images**
 - Out: Mon, Feb 9
 - Due: Sat, Feb 21 at 11:59pm

VARIATIONAL AUTOENCODERS

GAN → VAE → Diffusion

(in 5 minutes)

GANs

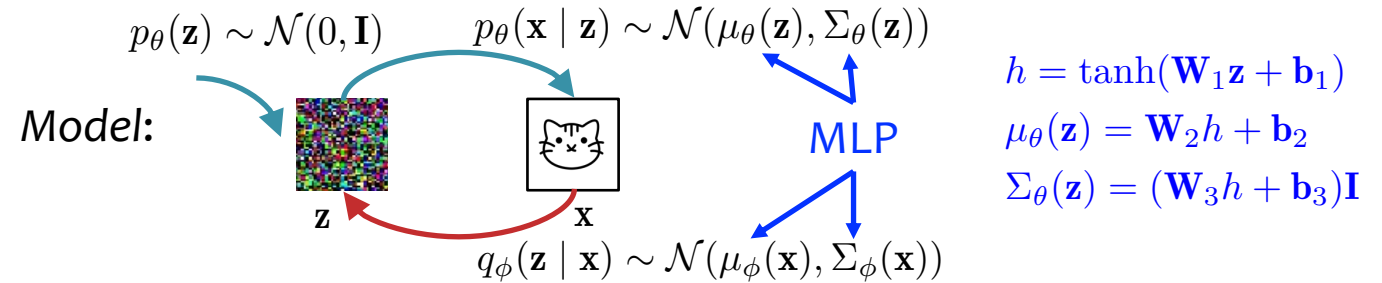


MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: minimax game

VAEs



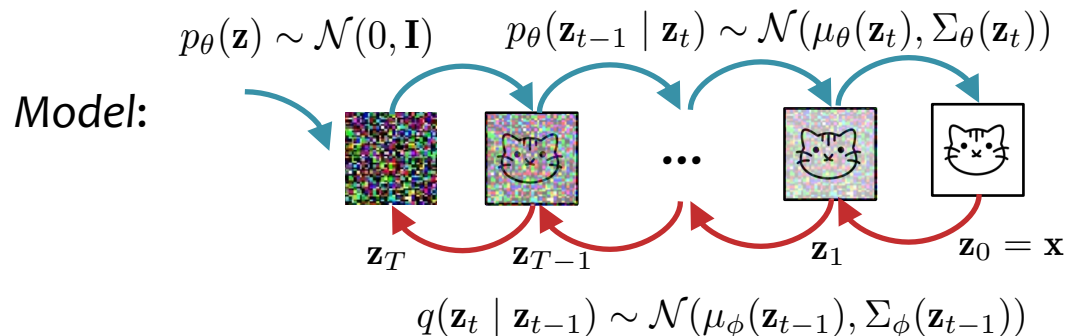
MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: $\theta, \phi = \arg \min_{\theta, \phi} \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p_{\theta}(\mathbf{z} | \mathbf{x}))$

where $p_{\theta}(\mathbf{z} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$

Diffusion



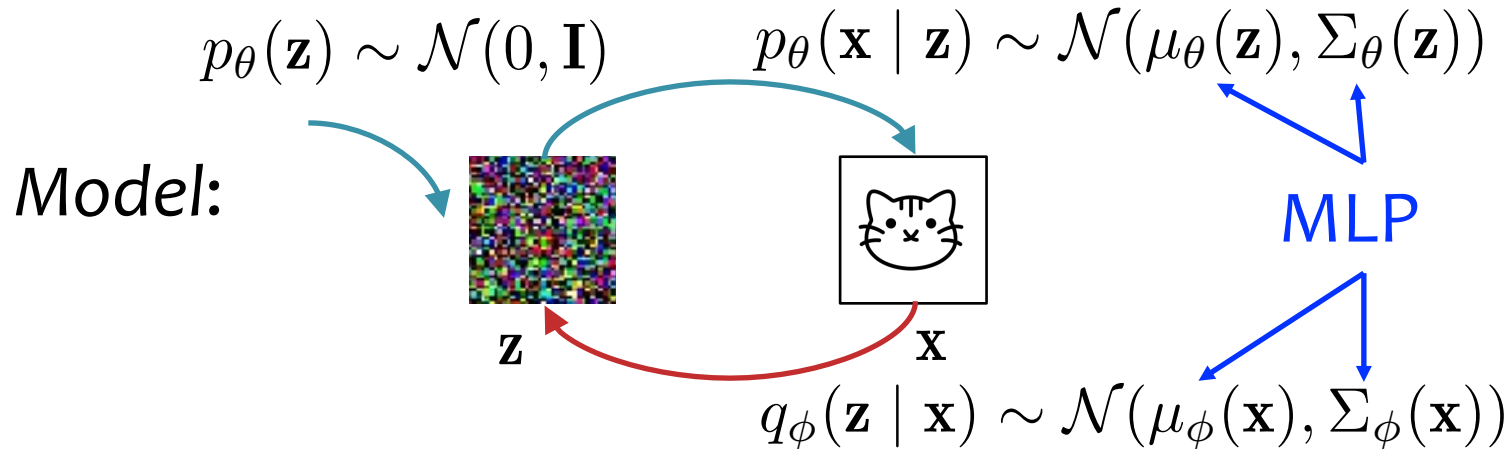
Learn: $\theta = \arg \min_{\theta} \text{KL}(q(\mathbf{z}_{1:T} | \mathbf{x}) \| p_{\theta}(\mathbf{z}_{1:T} | \mathbf{x}))$

where $p_{\theta}(\mathbf{z}_{1:T} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}_1) \cdots p(\mathbf{z}_{T-1} | \mathbf{z}_T) p(\mathbf{z}_T)$

MLE?: NOPE!

Variational Autoencoder

VAEs



$$h = \tanh(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1)$$
$$\mu_{\theta}(\mathbf{z}) = \mathbf{W}_2 h + \mathbf{b}_2$$
$$\Sigma_{\theta}(\mathbf{z}) = (\mathbf{W}_3 h + \mathbf{b}_3) \mathbf{I}$$

MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: $\theta, \phi = \arg \min_{\theta, \phi} \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p_{\theta}(\mathbf{z} | \mathbf{x}))$

where $p_{\theta}(\mathbf{z} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$

Implementing a VAE

Problem: How do we implement this?

Solution: It's easy, given a training image $\mathbf{x}^{(i)}$, we take $S = 1$ and do the following:

$$\Sigma_i = \text{diag}(\sigma)$$

① Let $\epsilon^{(1)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

② Compute enc. MLP: $\mu_\phi(\mathbf{x})$ and $\sigma_\phi(\mathbf{x})$

③ Compute latent: $\mathbf{z}^{(1)} = f(\epsilon^{(1)}) = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \epsilon^{(1)}$

④ Loss: $\ell(\theta, \phi) = \log p_\theta(\mathbf{x}^{(i)} | \mathbf{z}^{(1)}) - \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}^{(i)}) || p_\theta(\mathbf{z}))$

⑤ Backprop: $\nabla_{\theta, \phi} \ell(\theta, \phi)$

⑥ Gradient step: $\theta, \phi \leftarrow \theta, \phi - \gamma \nabla_{\theta, \phi} \ell(\theta, \phi)$

Background for Variational Autoencoders (VAEs)

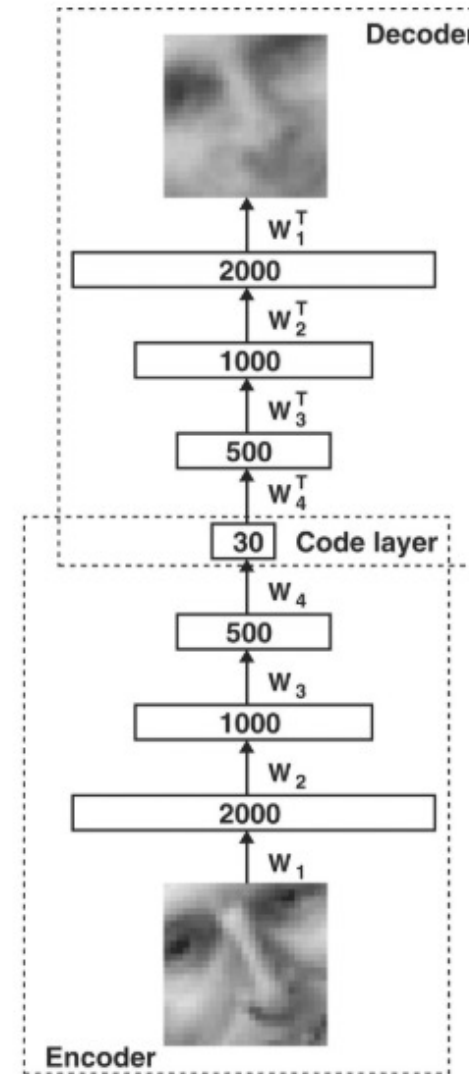
To talk about VAEs we need to lay some groundwork:

1. Autoencoders (not the variational kind)
2. KL divergence
3. Variational inference (using KL as an objective)
4. Monte Carlo estimation (for approximating expectations)
5. (Score function trick)
6. Reparameterization trick (for variance reduction)
7. Stochastic Gradient Variational Bayes

AUTOENCODERS (NOT THE VARIATIONAL KIND)

Autoencoders

- **Data:** only unlabeled instances $\mathbf{x}$
- **Model:** Autoencoders consist of two functions such that $|\mathbf{x}| = |\mathbf{x}'|$
 - $\mathbf{z} = \text{ENCODER}(\mathbf{x})$
 - $\mathbf{x}' = \text{DECODER}(\mathbf{z})$
- **Training:** minimize the reconstruction loss:
$$\|\mathbf{x} - \text{DECODER}(\text{ENCODER}(\mathbf{x}))\|^2$$
- **Goal:** to learn a low-dimensional representation of the inputs



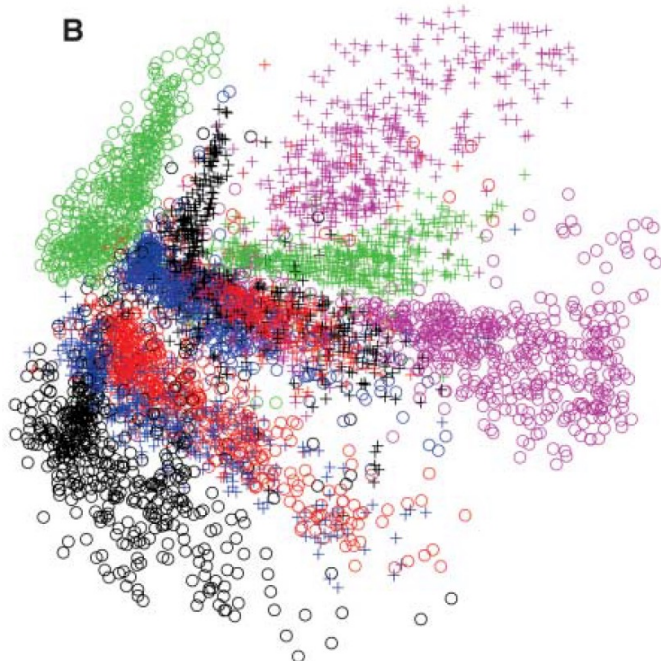
$\mathbf{x}'$, reconstruction

$\mathbf{z}$, latent rep.

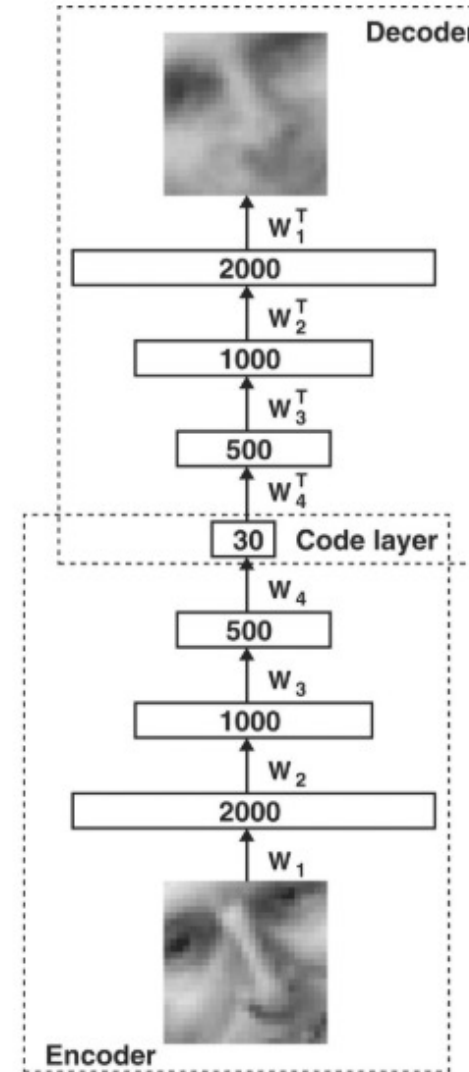
$\mathbf{x}$, input

Autoencoders

- **Problem:** the autoencoder does not define a probability distribution, so sampling from the latent space would effectively be sampling from the training instance reconstructions



autoencoder latent space



x' , reconstruction

z , latent rep.

x , input

Why VAEs?

- **Autoencoders:**
 - learn a low dimensional representation of the input, but hard to work with as a generative model
 - one of the key limitations of autoencoders is that we have no way of **sampling** from them!
- **Variational autoencoders (VAEs)**
 - by contrast learn a continuous latent space that is **easy to sample from!**
 - can **generate** new data (e.g. images) by sampling from the learned generative model

KL DIVERGENCE

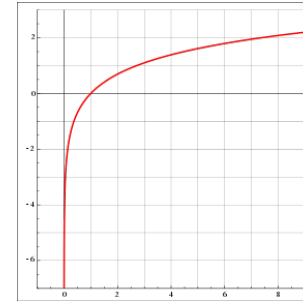
KL Divergence

- Definition: for two distributions $q(x)$ and $p(x)$ over $x \in \mathcal{X}$, the **KL Divergence** is:

$$\text{KL}(q||p) = E_{q(x)} \left[\log \frac{q(x)}{p(x)} \right] = \begin{cases} \sum_x q(x) \log \frac{q(x)}{p(x)} \\ \int_x q(x) \log \frac{q(x)}{p(x)} dx \end{cases}$$

- Properties:
 - $\text{KL}(q || p)$ measures the **proximity** of two distributions q and p
 - KL is **not** symmetric: $\text{KL}(q || p) \neq \text{KL}(p || q)$
 - KL is minimized when $q(x) = p(x)$ for all $x \in \mathcal{X}$

KL Divergence



Understanding the Behavior of KL as an objective function

Example 1: Keeping all else constant, consider the effect of a particular x' on $KL(q \parallel p)$

x'	$q(x')$	$p(x')$	$q(x') \log(q(x')/p(x'))$	effect on $KL(q \parallel p)$
1	0.4	0.4	0	no change
2	0.4	0.1	0.24	big increase
3	0.1	0.4	-0.06	little decrease
4	0.1	0.1	0	no change

KL **does** insist on good approximations for values that have **high** probability in q

KL **does not** insist on good approximations for values that have **low** probability in q

Example 2: Which q vector minimizes $KL(q \parallel p)$?

$$\underline{p} = \begin{bmatrix} 0.7 \\ 0.2 \\ 0.1 \end{bmatrix} \quad q^{(1)} = \begin{bmatrix} 1/3 \\ 1/3 \\ 1/3 \end{bmatrix} \quad \boxed{q^{(2)}} = \begin{bmatrix} 0.7 \\ 0.2 \\ 0.1 \end{bmatrix} \quad q^{(3)} = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}$$

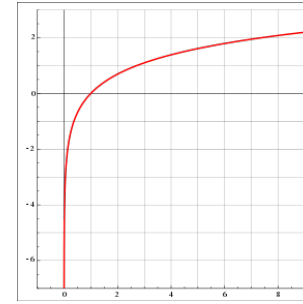
Q: If we're minimizing KL, why not return $q^{(3)}$?
A: Because it's not a distribution!

$$\text{KL}(q||p) = E_{q(x)} \left[\log \frac{q(x)}{p(x)} \right]$$

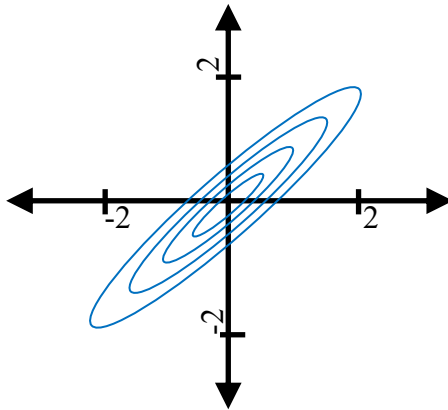
KL Divergence

Understanding the Behavior of KL as an objective function

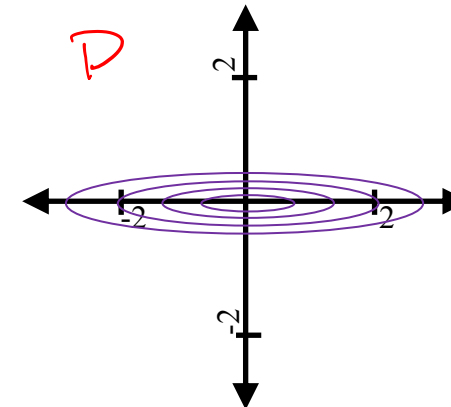
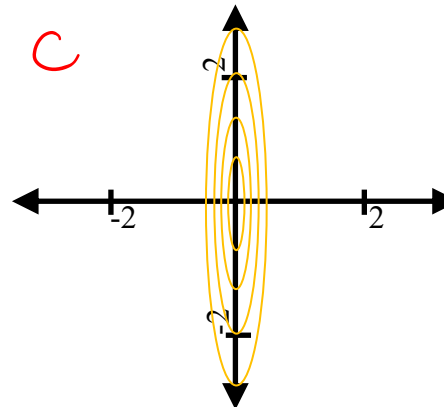
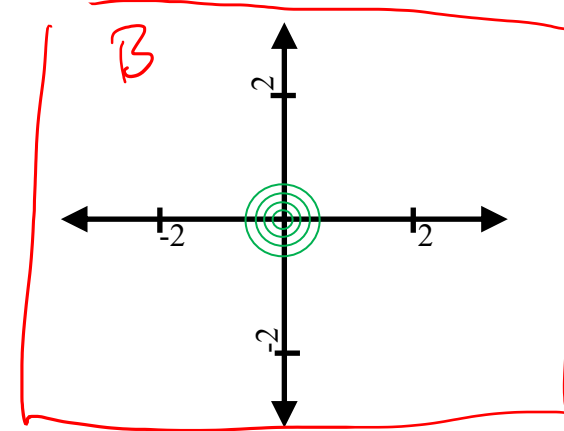
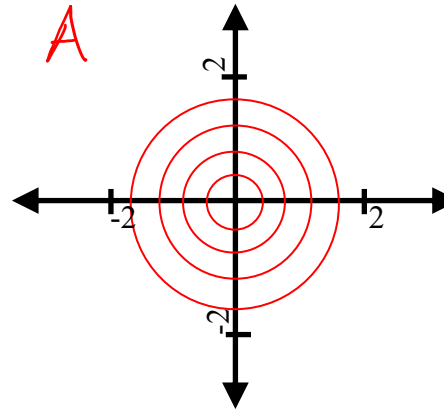
Example 3: Which q distribution minimizes $\text{KL}(q || p)$?



$$p(\mathbf{x}) = \mathcal{N}(\boldsymbol{\mu} = [0, 0]^T, \boldsymbol{\Sigma})$$



$$q(x_1, x_2) = \mathcal{N}_1(x_1 | \mu_1, \sigma_1^2) \mathcal{N}_2(x_2 | \mu_2, \sigma_2^2)$$



VARIATIONAL INFERENCE

Variational Inference Overview

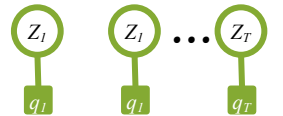
$$= p(x, z) / p(x)$$

1. Goal: estimate a distribution $p_\theta(z | x)$ given parameters θ
we assume this is intractable to compute exactly and $p(x)$ is #P-hard to compute
2. Variational Approximation: approx. with another distribution

$$q_\phi(z | x) \approx p_\theta(z | x)$$

Example: the *mean field approximation* assumes

$q_\phi(z | x) = \prod_t q_t(z_t | x; \phi)$ i.e., we decompose over variables



3. Optimization Problem: pick the q that minimizes $KL(q || p)$

$$\hat{\phi} = \operatorname{argmin}_{\phi \in \Theta} KL(q_\phi(z | x) || p_\theta(z | x))$$

4. Optimization Algorithm: various options

Example: gradient descent optimizes a surrogate objective $ELBO(q_\phi)$ to find ϕ

Optimizing KL Divergence

- Question: How do we minimize KL?

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \Theta} \underbrace{\text{KL}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\alpha}(\mathbf{z} \mid \mathbf{x}))}$$

- Answer #1: Oh no! We can't even compute this KL.

Why we can't compute KL...

$$\begin{aligned} \text{KL}(q(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z} \mid \mathbf{x})) &= E_{q(\mathbf{z} \mid \mathbf{x})} \left[\log \left(\frac{q(\mathbf{z} \mid \mathbf{x})}{p(\mathbf{z} \mid \mathbf{x})} \right) \right] \\ &= E_{q(\mathbf{z} \mid \mathbf{x})} [\log q(\mathbf{z} \mid \mathbf{x})] - E_{q(\mathbf{z} \mid \mathbf{x})} [\log p(\mathbf{z} \mid \mathbf{x})] \\ &= E_{q(\mathbf{z} \mid \mathbf{x})} [\log q(\mathbf{z} \mid \mathbf{x})] - E_{q(\mathbf{z} \mid \mathbf{x})} [\log p(\mathbf{x}, \mathbf{z})] + \boxed{E_{q(\mathbf{z} \mid \mathbf{x})} [\log p(\mathbf{x})]} \\ &= E_{q(\mathbf{z} \mid \mathbf{x})} [\log q(\mathbf{z} \mid \mathbf{x})] - E_{q(\mathbf{z} \mid \mathbf{x})} [\log p(\mathbf{x}, \mathbf{z})] + \log p(\mathbf{x}) \end{aligned}$$

Handwritten notes and annotations:

- A red arrow points from the term $\frac{q(\mathbf{z} \mid \mathbf{x})}{p(\mathbf{z} \mid \mathbf{x})}$ in the first line to the handwritten expression $\frac{p(\mathbf{x}, \mathbf{z})}{p(\mathbf{x})}$.
- A red arrow points from the term $E_{q(\mathbf{z} \mid \mathbf{x})} [\log p(\mathbf{z} \mid \mathbf{x})]$ in the second line to the boxed term $E_{q(\mathbf{z} \mid \mathbf{x})} [\log p(\mathbf{x})]$ in the fourth line.
- A green bracket underlines the first two terms of the final equation, with the text "this expectation does not depend on q" written below it.
- A red bracket underlines the last term of the final equation, with the text "we assumed this is intractable to compute!" written below it.

Optimizing KL Divergence

- Question: How do we minimize KL?

$$\hat{\theta} = \operatorname{argmin}_{\theta \in \Theta} \underbrace{\operatorname{KL}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\alpha}(\mathbf{z} \mid \mathbf{x}))}_{\text{KL Divergence}}$$

- Answer #2: We don't need to compute this KL
We can instead maximize the ELBO (i.e. **E**vidence **L**ower **B**ound)

$$\operatorname{ELBO}(q_{\theta}) = E_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log p_{\alpha}(\mathbf{x}, \mathbf{z})] - E_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log q_{\theta}(\mathbf{z} \mid \mathbf{x})]$$

The ELBO for a DGM

Here is why...

$$\begin{aligned} \theta &= \operatorname{argmin}_{\theta} \operatorname{KL}(q_{\theta}(\mathbf{z} \mid \mathbf{x}) \parallel p_{\alpha}(\mathbf{z} \mid \mathbf{x})) \leftarrow \\ &= \operatorname{argmin}_{\theta} E_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log q_{\theta}(\mathbf{z} \mid \mathbf{x})] - E_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log p_{\alpha}(\mathbf{x}, \mathbf{z})] + \log p_{\alpha}(\mathbf{x}) \\ &= \operatorname{argmin}_{\theta} E_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log q_{\theta}(\mathbf{z} \mid \mathbf{x})] - E_{q_{\theta}(\mathbf{z} \mid \mathbf{x})} [\log p_{\alpha}(\mathbf{x}, \mathbf{z})] \\ &= \operatorname{argmax}_{\theta} \operatorname{ELBO}(q_{\theta}) \end{aligned}$$

dropping the
intractable term
gives the ELBO

ELBO as Objective Function

What does maximizing $\text{ELBO}(q_\phi)$ accomplish?

$$\text{ELBO}(q_\theta) = E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log p_\alpha(\mathbf{x}, \mathbf{z})] - E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log q_\theta(\mathbf{z} | \mathbf{x})]$$

1. The first expectation is high if q_ϕ puts probability mass on the same values of $\mathbf{z}$ that p_θ puts probability mass

2. The second term is the entropy of q_ϕ and the entropy will be high if q_ϕ spreads its probability mass evenly

$= E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log p(\mathbf{x}|\mathbf{z})] - \text{KL}(q_\theta(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z}))$

ELBO as lower bound

Theorem: For any q , $\log p(\mathbf{x}) \geq \text{ELBO}(q)$
i.e. $\text{ELBO}(q)$ is a lower bound for $\log p(\mathbf{x})$

Note:

$$\begin{aligned}\text{ELBO}(q_\theta) &= \left(E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log p_\alpha(\mathbf{x}, \mathbf{z})] - E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log q_\theta(\mathbf{z} | \mathbf{x})] \right) \\ \text{KL}(q_\theta(\mathbf{z} | \mathbf{x}) \parallel p_\alpha(\mathbf{z} | \mathbf{x})) &= \left(E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log q_\theta(\mathbf{z} | \mathbf{x})] - E_{q_\theta(\mathbf{z}|\mathbf{x})} [\log p_\alpha(\mathbf{x}, \mathbf{z})] \right) + \log p_\alpha(\mathbf{x})\end{aligned}$$

Proof #1:

1. $\log p(\mathbf{x}) = \text{KL}(q \parallel p) + \text{ELBO}(q)$
2. $\text{KL}(q \parallel p) \geq 0$
3. $\log p(\mathbf{x}) \geq \text{ELBO}(q)$

Takeaways:

1. in variational inference, we find the q that gives the **tightest bound** on the normalization constant for $p(\mathbf{z} | \mathbf{x})$
2. maximizing the ELBO is equivalent to minimizing KL
3. maximizing the ELBO is maximizing a lower bound on the likelihood $p(\mathbf{x})$

MONTE CARLO ESTIMATION

Monte Carlo Estimation

Goal: Estimate the expectation of a function f of a (possibly high dimensional) random variable $\mathbf{X} \sim P(\mathbf{X})$:

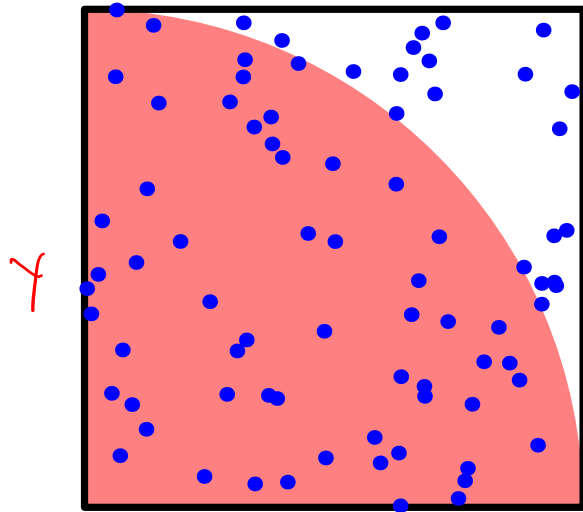
$$\phi = \mathbb{E}_{\mathbf{x} \sim p(\mathbf{x})} [f(\mathbf{x})] = \int_{\mathbf{x}} p(\mathbf{x}) f(\mathbf{x}) d\mathbf{x}$$

Monte Carlo Estimate: Approximate the expectation by drawing S samples from p and computing the average of the function f on the samples:

$$\hat{\phi} = \frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)}) \text{ where } \mathbf{x}^{(1)}, \dots, \mathbf{x}^{(S)} \stackrel{\text{i.i.d.}}{\sim} p(\mathbf{x})$$

Monte Carlo Estimation

Example: A Dumb Approximation of π

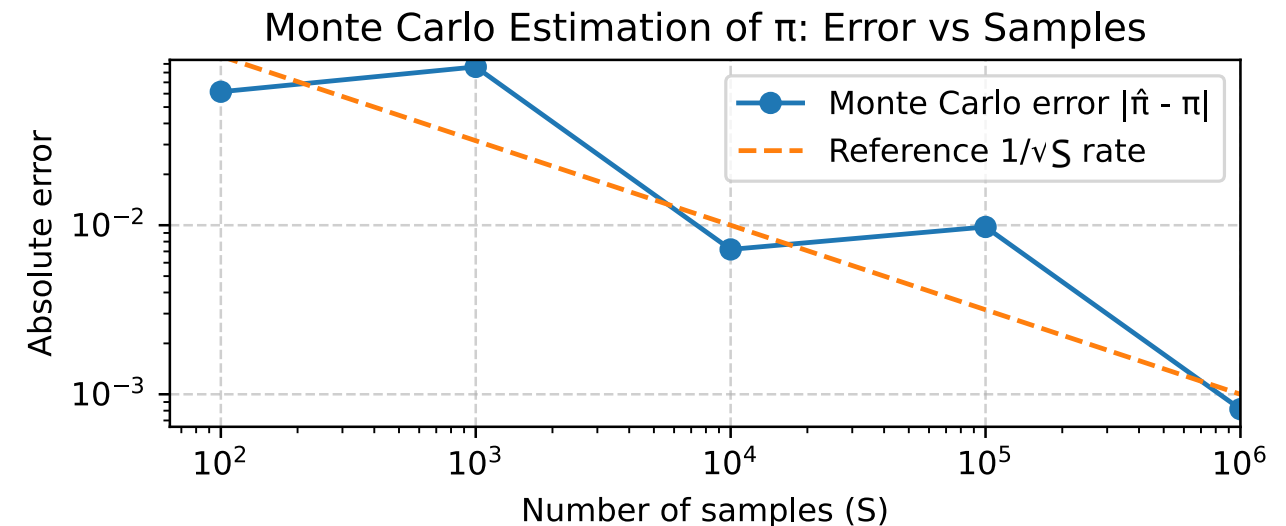


$$P(x, y) = \begin{cases} 1 & 0 < x < 1 \text{ and } 0 < y < 1 \\ 0 & \text{otherwise} \end{cases}$$

$$f(x, y) = 4 \mathbb{I}(x^2 + y^2 < 1)$$

$$\pi = 4 \iint \mathbb{I}((x^2 + y^2) < 1) P(x, y) \, dx \, dy$$

```
$ python monte-carlo-pi.py
S=100      pi ≈ 3.080000  error=0.061593
S=1000     pi ≈ 3.228000  error=0.086407
S=10000    pi ≈ 3.134400  error=0.007193
S=100000   pi ≈ 3.151360  error=0.009767
S=1000000  pi ≈ 3.140776  error=0.000817
```



Monte Carlo Estimation

Properties:

The Monte Carlo estimate is an **unbiased estimator**:

$$\begin{aligned}\mathbb{E}[\hat{\phi}] &= \mathbb{E}\left[\frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)})\right] = \frac{1}{S} \sum_{s=1}^S \mathbb{E}[f(\mathbf{x}^{(s)})] \\ &= \mathbb{E}[f(\mathbf{x})] = \phi\end{aligned}$$

And the **variance** of $\hat{\phi}$ shrinks as σ^2/S where σ^2 is the variance of ϕ :

$$\begin{aligned}\text{Var}(\hat{\phi}) &= \text{Var}\left(\frac{1}{S} \sum_{s=1}^S f(\mathbf{x}^{(s)})\right) = \frac{1}{S^2} \sum_{s=1}^S \text{Var}(f(\mathbf{x}^{(s)})) \\ &= \frac{1}{S^2} \cdot S \cdot \text{Var}(f(\mathbf{x})) = \frac{\sigma^2}{S}\end{aligned}$$

(So the **standard error** shrinks as $\sqrt{\sigma^2/S}$.)

Practical Challenges:

The convergence rate is independent of the dimensionality of $\mathbf{x}$, which is great!

...but the variance constant σ^2 could be impractically large for high dimensional problems.

REPARAMETERIZATION TRICK

Reparameterization Trick (1D Gaussian)

- **Problem:** you want to backprop through a sample from a Gaussian, but you can't (at first glance) because it's stochastic
- **Solution:** reparametrize the sampling process

Suppose the variational distribution is

$$q_{\phi}(z \mid x) = \mathcal{N}(z; \mu_{\phi}(x), \sigma_{\phi}^2(x)).$$

Instead of sampling z directly, we reparameterize as

$$z = \mu_{\phi}(x) + \sigma_{\phi}(x) \epsilon, \quad \epsilon \sim \mathcal{N}(0, 1).$$

This expresses z as a differentiable function of parameters $\mu_{\phi}(x), \sigma_{\phi}(x)$ and independent noise ϵ , allowing gradients to flow through z during training.

Reparameterization Trick (Multivariate Gaussian)

- **Problem:** you want to backprop through a sample from a Gaussian, but you can't (at first glance) because it's stochastic
- **Solution:** reparametrize the sampling process

Suppose the variational distribution is

$$q_{\phi}(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\mathbf{z}; \boldsymbol{\mu}_{\phi}(\mathbf{x}), \boldsymbol{\Sigma}_{\phi}(\mathbf{x})), \quad \boldsymbol{\Sigma}_{\phi} = \text{diag}(\boldsymbol{\sigma}_{\phi}^2). \quad = \quad \sigma_{\phi}^2 \mathbf{I}$$

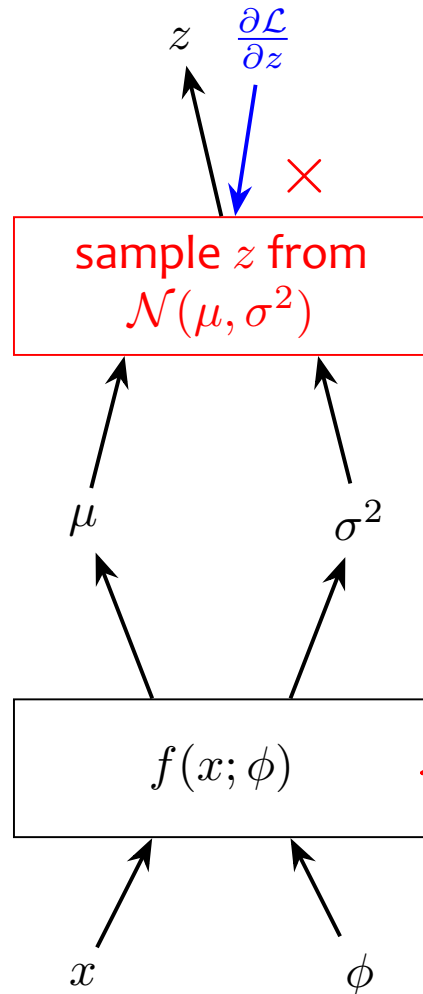
Instead of sampling $\mathbf{z}$ directly, we reparameterize as

$$\mathbf{z} = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \boldsymbol{\epsilon}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, \mathbf{I}).$$

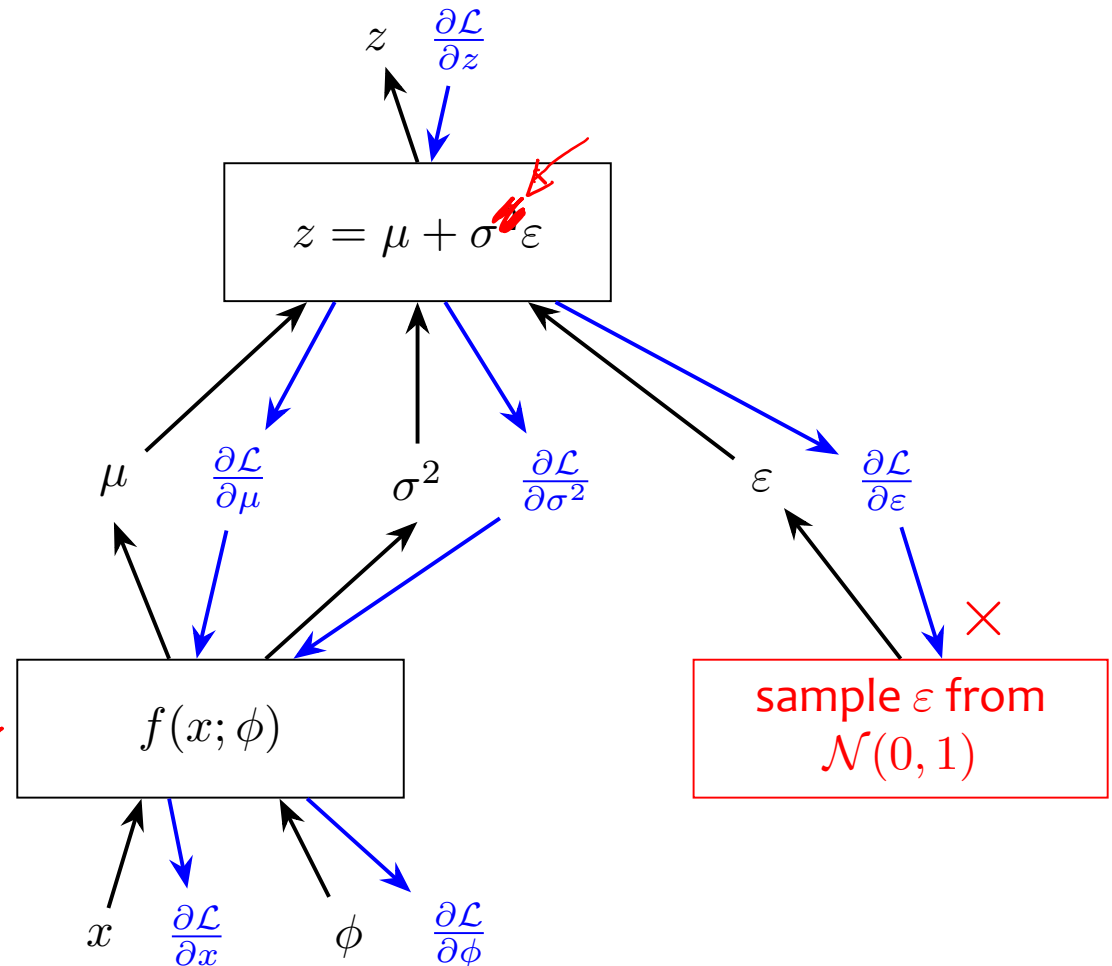
This expresses $\mathbf{z}$ as a differentiable function of parameters $\boldsymbol{\mu}_{\phi}(\mathbf{x})$, $\boldsymbol{\sigma}_{\phi}(\mathbf{x})$ and independent noise $\boldsymbol{\epsilon}$, allowing gradients to flow through $\mathbf{z}$ during training.

Reparameterization Trick (1D Gaussian)

without reparameterization



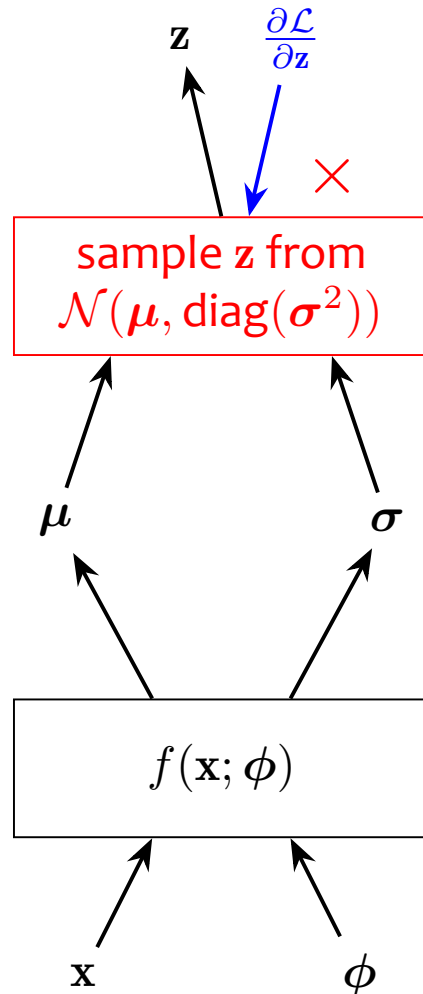
with reparameterization



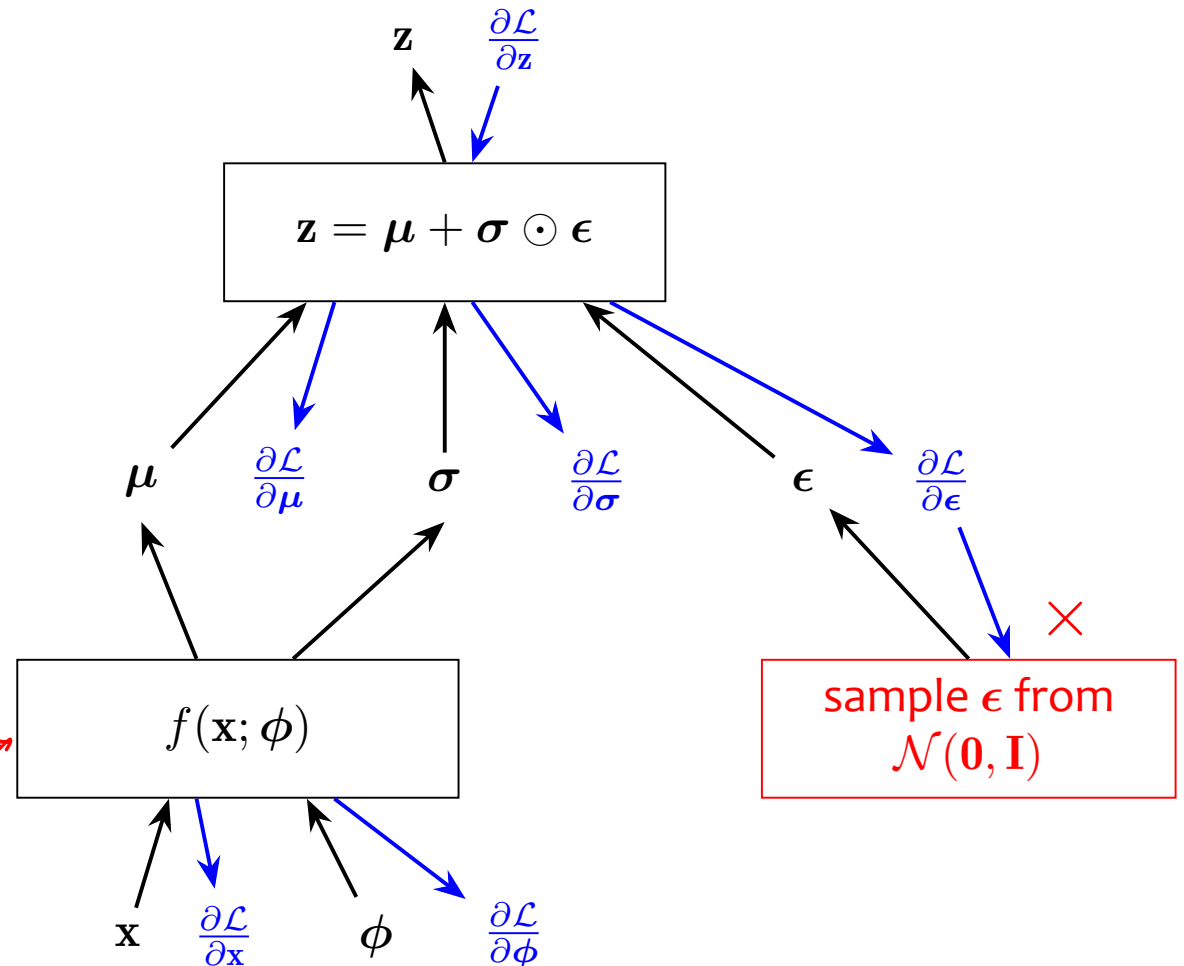
MLP

Reparameterization Trick (Multivariate Gaussian)

without reparameterization



with reparameterization



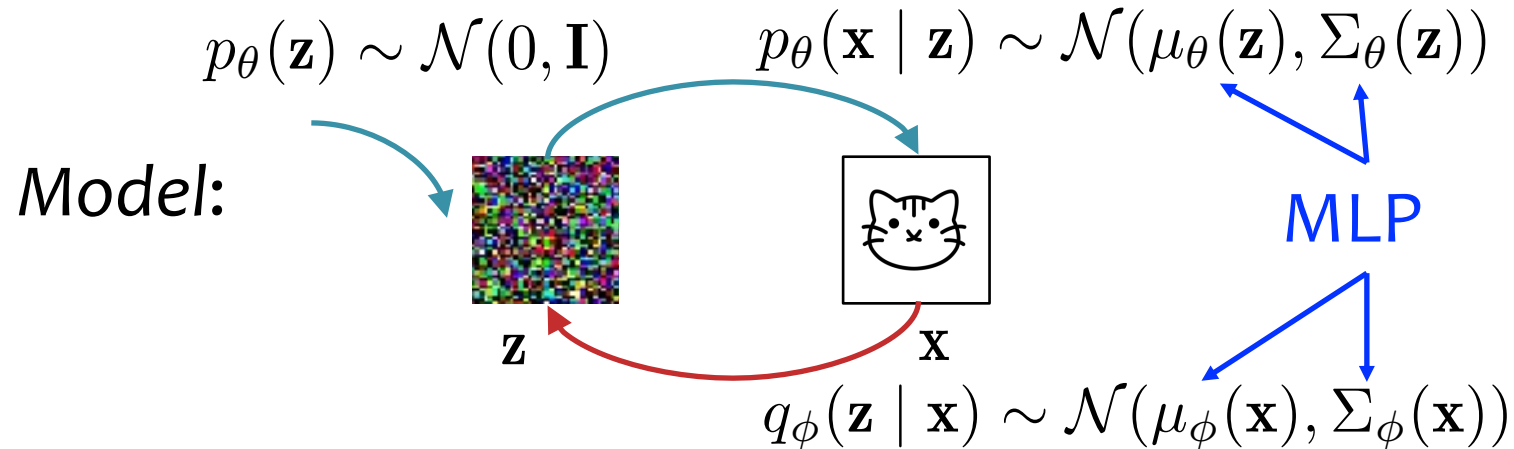
← MLP



VARIATIONAL AUTOENCODERS

Variational Autoencoder

VAEs



$$h = \tanh(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1)$$
$$\mu_{\theta}(\mathbf{z}) = \mathbf{W}_2 h + \mathbf{b}_2$$
$$\Sigma_{\theta}(\mathbf{z}) = (\mathbf{W}_3 h + \mathbf{b}_3) \mathbf{I}$$

MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: $\theta, \phi = \arg \min_{\theta, \phi} \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p_{\theta}(\mathbf{z} | \mathbf{x}))$

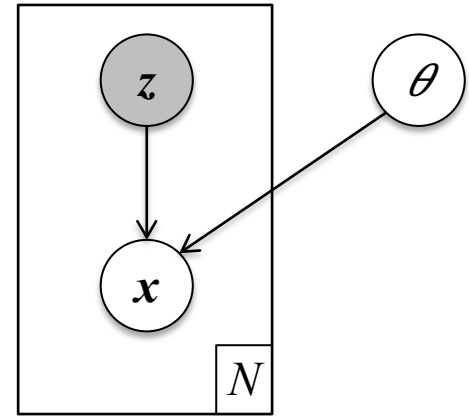
where $p_{\theta}(\mathbf{z} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$

Variational Autoencoder

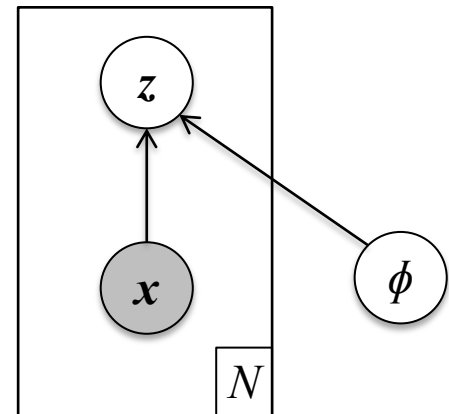
Graphical Model Perspective

- We can view a variational autoencoder (VAE) as consisting of two graphical models; each of which is defined by a neural network
- VAEs define two distributions:
 - **encoder:** $q_{\phi}(\mathbf{z} \mid \mathbf{x})$
 - **decoder:** $p_{\theta}(\mathbf{x} \mid \mathbf{z})$
- Parameters θ and ϕ are neural network parameters (i.e. θ are not the variational parameters)

$$p_{\theta}(\mathbf{x} \mid \mathbf{z})$$



$$q_{\phi}(\mathbf{z} \mid \mathbf{x})$$

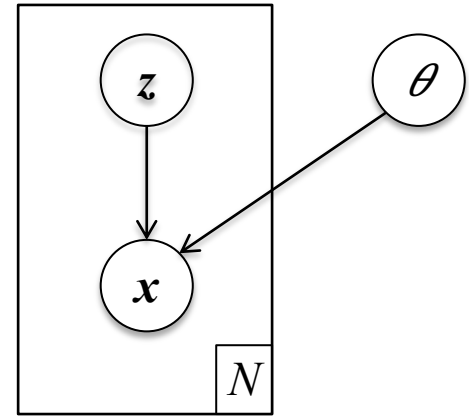


Variational Autoencoder

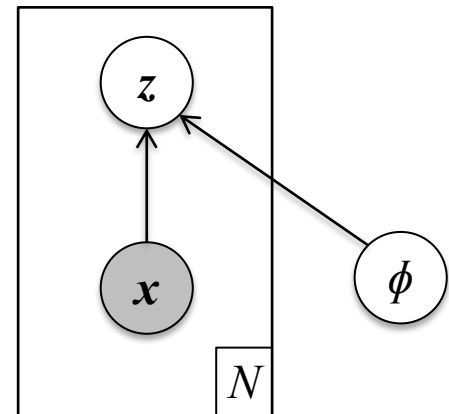
Graphical Model Perspective

- We can view a variational autoencoder (VAE) as consisting of two graphical models; each of which is defined by a neural network
- VAEs define two distributions:
 - **encoder:** $q_{\phi}(\mathbf{z} \mid \mathbf{x})$
 - **decoder:** $p_{\theta}(\mathbf{x} \mid \mathbf{z})$
- Parameters θ and ϕ are neural network parameters (i.e. θ are not the variational parameters)
- The decoder is actually a joint model over both $\mathbf{x}$ and $\mathbf{z}$.
 - **joint decoder model:**
 $p_{\theta}(\mathbf{x}, \mathbf{z}) = p_{\theta}(\mathbf{x} \mid \mathbf{z}) p(\mathbf{z})$

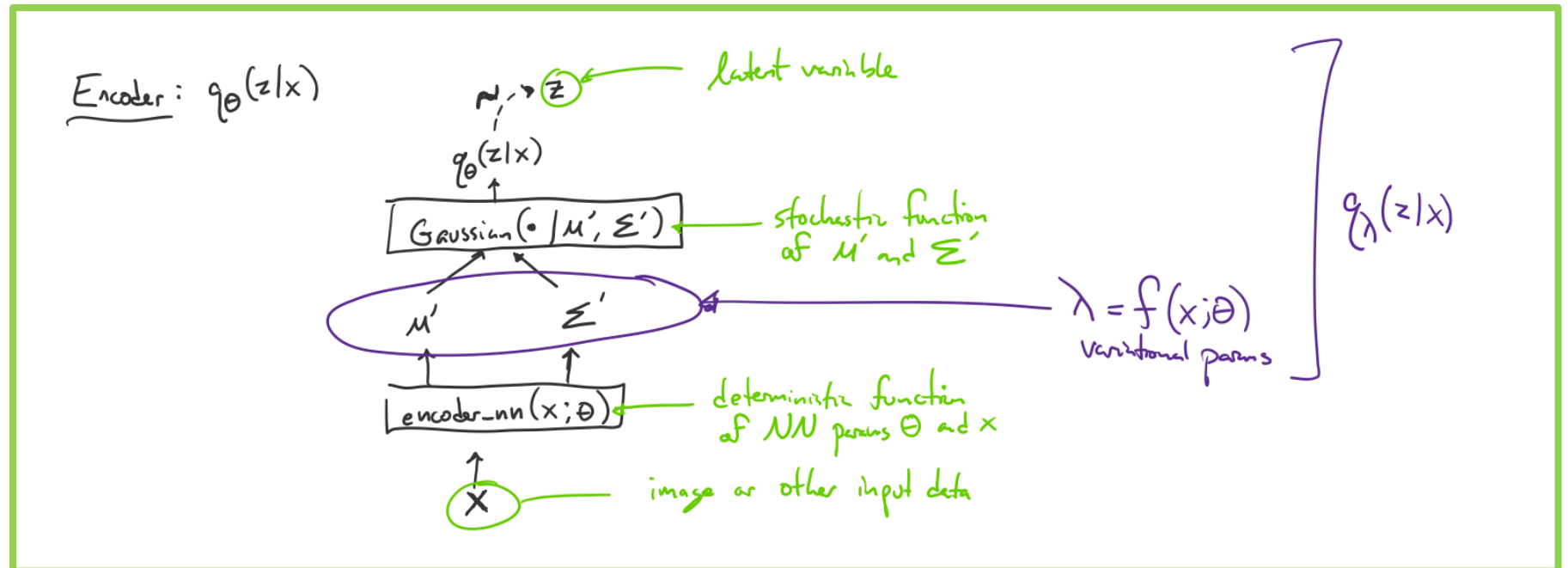
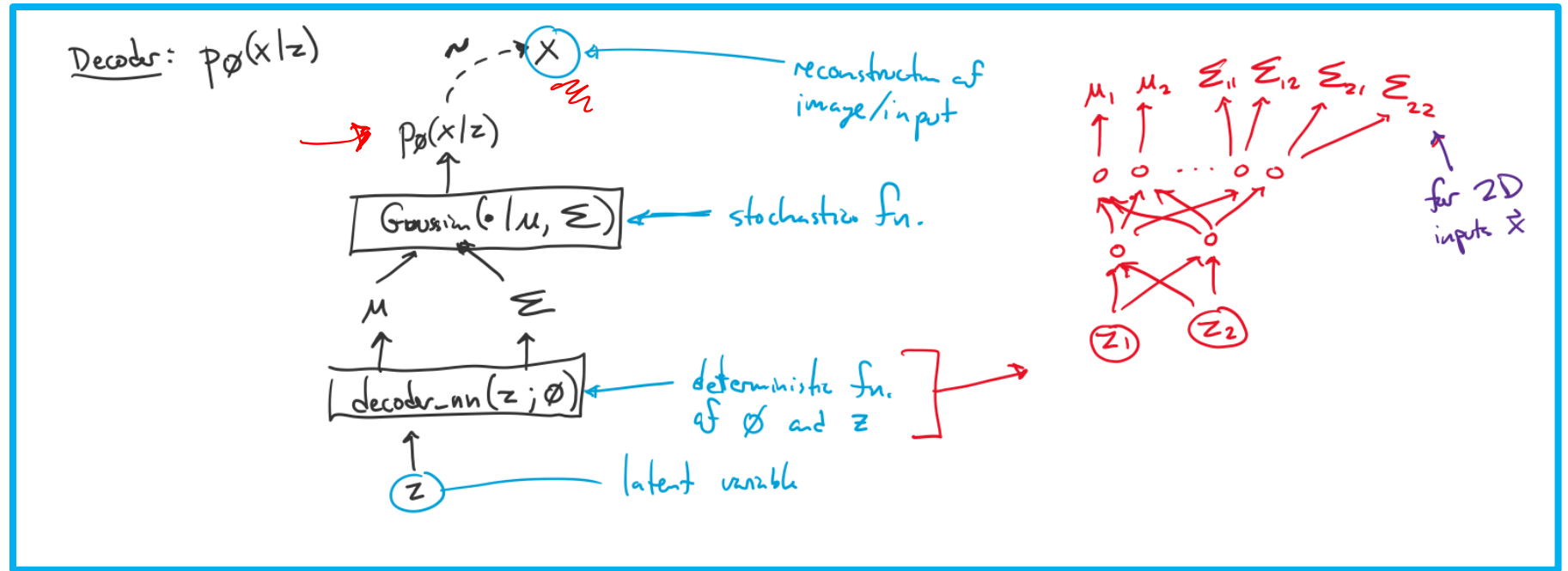
$$p_{\theta}(\mathbf{x}, \mathbf{z})$$



$$q_{\phi}(\mathbf{z} \mid \mathbf{x})$$



VAEs: Neural Network View



VAE Reparameterization Trick

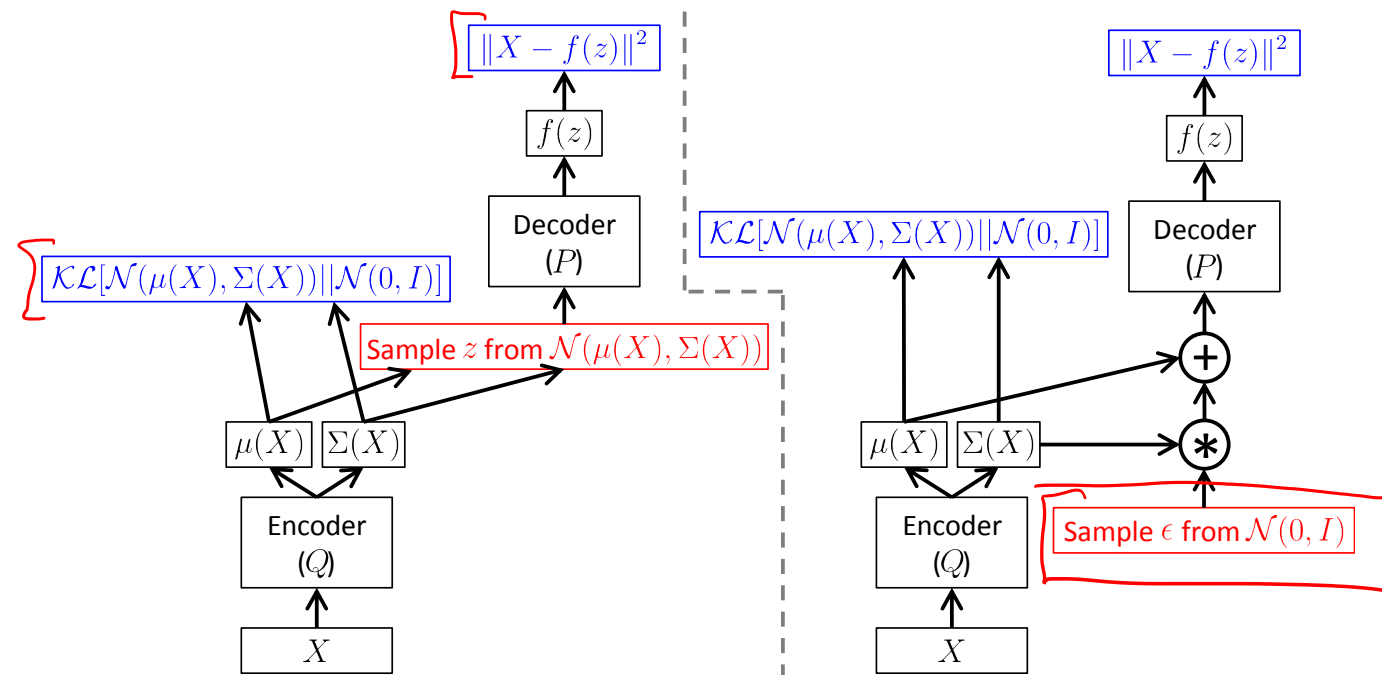


Figure 4: A training-time variational autoencoder implemented as a feed-forward neural network, where $P(X|z)$ is Gaussian. Left is without the “reparameterization trick”, and right is with it. Red shows sampling operations that are non-differentiable. Blue shows loss layers. The feedforward behavior of these networks is identical, but backpropagation can be applied only to the right network.

Background for Variational Autoencoders (VAEs)

To talk about VAEs we need to lay some groundwork:

1. Autoencoders (not the variational kind)
2. KL divergence
3. Variational inference (using KL as an objective)
4. Monte Carlo estimation (for approximating expectations)
5. (Score function trick)
6. Reparameterization trick (for variance reduction)
7. Stochastic Gradient Variational Bayes

Deriving the Variational Autoencoder

1. *Problem:* Cannot sample from an autoencoder.

Solution: Define a decoder that we can sample from:

$$p_{\theta}(\mathbf{z}) \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$p_{\theta}(\mathbf{x} | \mathbf{z}) \sim \mathcal{N}(\mu_{\theta}(\mathbf{z}), \Sigma_{\theta}(\mathbf{z}))$ where $\mu_{\theta}(\mathbf{z})$ and $\Sigma_{\theta}(\mathbf{z})$ are neural nets.

2. *Problem:* Intractable to maximize data likelihood with this decoder:

$$\hat{\theta} = \operatorname{argmax}_{\theta} \log p_{\theta}(\mathbf{x}) = \operatorname{argmax}_{\theta} \int_{\mathbf{z}} p_{\theta}(\mathbf{x} | \mathbf{z}) p_{\theta}(\mathbf{z}) d\mathbf{z}$$

Solution: Maximize a lower bound (ELBO) instead by introducing an encoder distribution:

$$q_{\phi}(\mathbf{z} | \mathbf{x}) \sim \mathcal{N}(\mu_{\phi}(\mathbf{x}), \Sigma_{\phi}(\mathbf{x}))$$

$$\hat{\theta}, \hat{\phi} = \operatorname{argmax}_{\theta, \phi} \operatorname{ELBO}(q_{\phi}, p_{\theta}) \text{ where } \log p_{\theta}(\mathbf{x}) \geq \operatorname{ELBO}(q_{\phi}, p_{\theta})$$

Deriving the Variational Autoencoder

3. *Problem:* Intuitively, we should alternately maximize ϕ to match q_ϕ to the current p_θ as close as possible in KL (variational inference), and then maximize θ to push up the likelihood $\log p_\theta(\mathbf{x})$ as much as possible (learning); but this is slow.

Solution: Instead, use SGA and maximize θ and ϕ simultaneously:

$$(\theta, \phi) \leftarrow (\theta, \phi) + \gamma \nabla_{\theta, \phi} \text{ELBO}(q_\phi, p_\theta)$$

4. *Problem:* Cannot compute the first expectation in $\text{ELBO}(q_\phi, p_\theta)$:
 $\text{ELBO}(q_\phi, p_\theta) = E_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} | \mathbf{z})] - \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}) \| p_\theta(\mathbf{z}))$

Solution: Approximate with Monte Carlo estimation:

$$E_{q_\phi(\mathbf{z}|\mathbf{x})}[\log p_\theta(\mathbf{x} | \mathbf{z})] \approx \frac{1}{S} \sum_{s=1}^S \log p_\theta(\mathbf{x} | \mathbf{z}^{(s)}) \text{ where } \mathbf{z}^{(s)} \sim q_\phi(\mathbf{z} | \mathbf{x})$$

Deriving the Variational Autoencoder

5. *Problem:* This approximation has very high variance, a known problem in SGVB (cf. score matching trick):

$$\nabla_{\phi} E_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z})] = E_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z}) \nabla_{\phi} \log q_{\phi}(\mathbf{z} | \mathbf{x})]$$

Solution: Apply reparameterization trick, then Monte Carlo estimate:

Let $\epsilon^{(s)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ and set $\mathbf{z}^{(s)} = f(\epsilon^{(s)}) = \boldsymbol{\mu}_{\phi}(\mathbf{x}) + \boldsymbol{\sigma}_{\phi}(\mathbf{x}) \odot \epsilon^{(s)}$
 ~~$\epsilon^{(s)}$~~ .

$$\begin{aligned} E_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z})] &= E_{\epsilon \sim \mathcal{N}} [\log p_{\theta}(\mathbf{x} | \mathbf{z} = f(\epsilon))] \\ &\approx \frac{1}{S} \sum_{s=1}^S \log p_{\theta}(\mathbf{x} | \mathbf{z}^{(s)}) \end{aligned}$$

Deriving the Variational Autoencoder

6. *Problem:* How do we implement this?

Solution: It's easy, given a training image $\mathbf{x}^{(i)}$, we take $S = 1$ and do the following:

Let $\epsilon^{(1)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$

Compute enc. MLP: $\mu_\phi(\mathbf{x})$ and $\sigma_\phi(\mathbf{x})$

Compute latent: $\mathbf{z}^{(1)} = f(\epsilon^{(1)}) = \mu_\phi(\mathbf{x}) + \sigma_\phi(\mathbf{x}) \odot \epsilon^{(1)}$

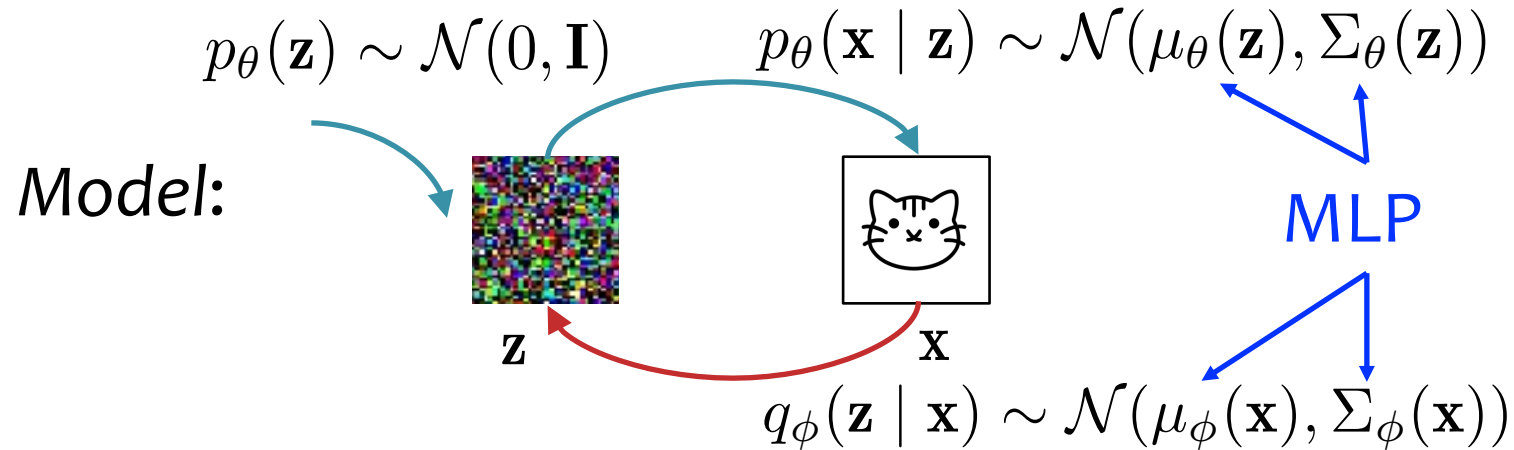
Loss: $\ell(\theta, \phi) = \underbrace{\log p_\theta(\mathbf{x}^{(i)} | \mathbf{z}^{(1)})}_{\text{reconstruction loss}} - \text{KL}(q_\phi(\mathbf{z} | \mathbf{x}^{(i)}) || p_\theta(\mathbf{z}))$

Backprop: $\nabla_{\theta, \phi} \ell(\theta, \phi)$

Gradient step: $\theta, \phi \leftarrow \theta, \phi - \gamma \nabla_{\theta, \phi} \ell(\theta, \phi)$

Variational Autoencoder

VAEs



$$h = \tanh(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1)$$
$$\mu_{\theta}(\mathbf{z}) = \mathbf{W}_2 h + \mathbf{b}_2$$
$$\Sigma_{\theta}(\mathbf{z}) = (\mathbf{W}_3 h + \mathbf{b}_3) \mathbf{I}$$

MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: $\theta, \phi = \arg \min_{\theta, \phi} \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) \| p_{\theta}(\mathbf{z} | \mathbf{x}))$

where $p_{\theta}(\mathbf{z} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$

Why VAEs?

- **Autoencoders:**
 - learn a low dimensional representation of the input, but hard to work with as a generative model
 - one of the key limitations of autoencoders is that we have no way of **sampling** from them!
- **Variational autoencoders (VAEs)**
 - by contrast learn a continuous latent space that is **easy to sample from!**
 - can **generate** new data (e.g. images) by sampling from the learned generative model

VAE RESULTS

VAEs for Image Generation

Kingma & Welling (2014)

- introduced VAEs
- applied to image generation

Model

- $p_{\phi}(\mathbf{z}) \sim N(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- $p_{\phi}(\mathbf{x} | \mathbf{z})$ is a multivariate Gaussian with mean and variance computed by an MLP, fully connected neural network with a single hidden layer with parameters ϕ
- $q_{\theta}(\mathbf{z} | \mathbf{x})$ is a multivariate Gaussian with diagonal covariance structure and with mean and variance computed by an MLP with parameters θ

Auto-Encoding Variational Bayes

Diederik P. Kingma
Machine Learning Group
Universiteit van Amsterdam
dpkingma@gmail.com

Max Welling
Machine Learning Group
Universiteit van Amsterdam
welling.max@gmail.com

Abstract

How can we perform efficient inference and learning in directed probabilistic models, in the presence of continuous latent variables with intractable posterior distributions, and large datasets? We introduce a stochastic variational inference and learning algorithm that scales to large datasets and, under some mild differentiability conditions, even works in the intractable case. Our contributions is two-fold. First, we show that a reparameterization of the variational lower bound yields a lower bound estimator that can be straightforwardly optimized using standard stochastic gradient methods. Second, we show that for i.i.d. datasets with continuous latent variables per datapoint, posterior inference can be made especially efficient by fitting an approximate inference model (also called a recognition model) to the intractable posterior using the proposed lower bound estimator. Theoretical advantages are reflected in experimental results.

VAEs for Image Generation

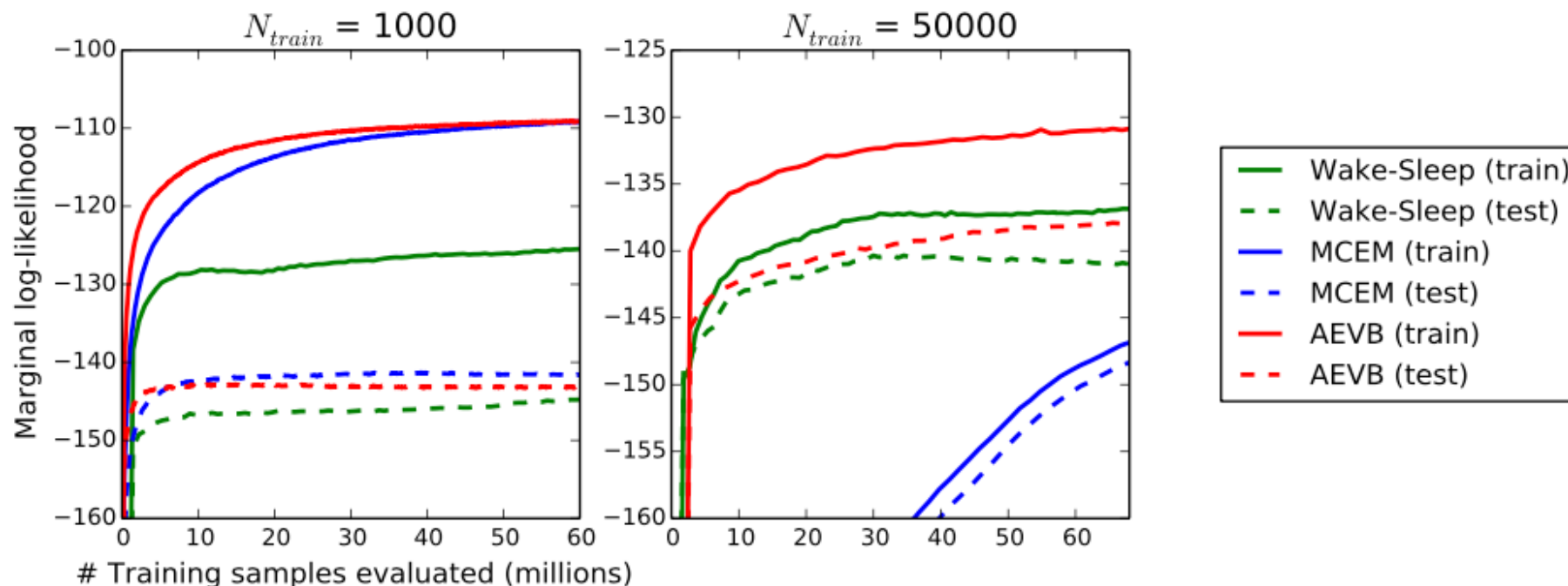
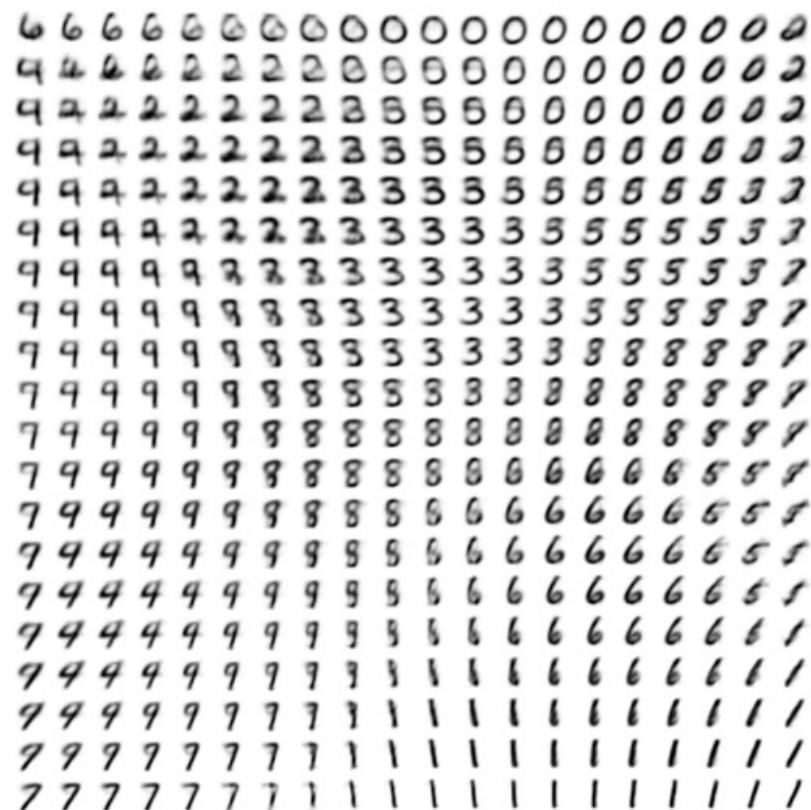


Figure 3: Comparison of AEVB to the wake-sleep algorithm and Monte Carlo EM, in terms of the estimated marginal likelihood, for a different number of training points. Monte Carlo EM is not an on-line algorithm, and (unlike AEVB and the wake-sleep method) can't be applied efficiently for the full MNIST dataset.

VAEs for Image Generation



(a) Learned Frey Face manifold



(b) Learned MNIST manifold

Figure 4: Visualisations of learned data manifold for generative models with two-dimensional latent space, learned with AEVB. Since the prior of the latent space is Gaussian, linearly spaced coordinates on the unit square were transformed through the inverse CDF of the Gaussian to produce values of the latent variables $\mathbf{z}$. For each of these values $\mathbf{z}$, we plotted the corresponding generative $p_{\theta}(\mathbf{x}|\mathbf{z})$ with the learned parameters θ .

VAEs for Image Generation

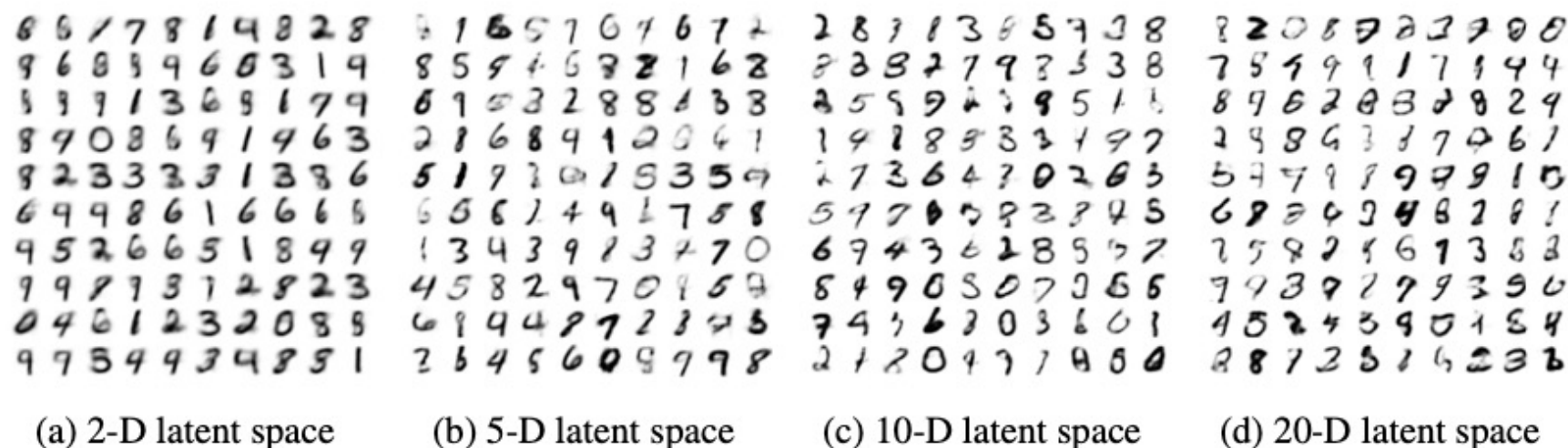


Figure 5: Random samples from learned generative models of MNIST for different dimensionalities of latent space.

VAEs for Text Generation

Bowman et al. (2015)

- example of an application of VAEs to discrete data
- built on the sequence-to-sequence framework:
 - input is read in by an LSTM
 - output is generated by an LSTM-LM

Model

- $p_{\phi}(\mathbf{z}) \sim N(\mathbf{z}; \mathbf{0}, \mathbf{I})$
- $p_{\phi}(\mathbf{x} | \mathbf{z})$ is an LSTM Language Model with parameters ϕ
- $q_{\theta}(\mathbf{z} | \mathbf{x})$ is a multivariate Gaussian with mean and variance computed by an LSTM with parameters θ

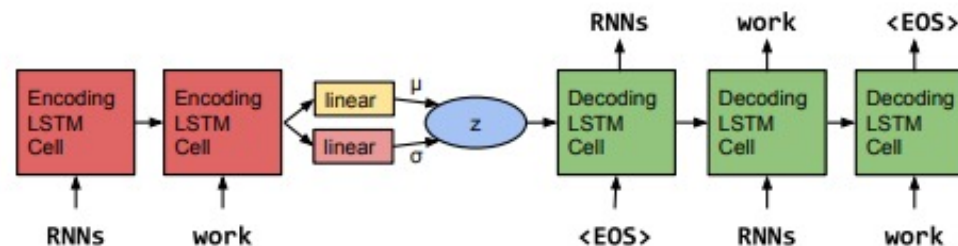


Figure 1: The core structure of our variational autoencoder language model. Words are represented using a learned dictionary of embedding vectors.

VAEs for Text Generation

INPUT	we looked out at the setting sun .	i went to the kitchen .	how are you doing ?
MEAN	<i>they were laughing at the same time .</i>	<i>i went to the kitchen .</i>	<i>what are you doing ?</i>
SAMP. 1	<i>ill see you in the early morning .</i>	<i>i went to my apartment .</i>	<i>“ are you sure ?</i>
SAMP. 2	<i>i looked up at the blue sky .</i>	<i>i looked around the room .</i>	<i>what are you doing ?</i>
SAMP. 3	<i>it was down on the dance floor .</i>	<i>i turned back to the table .</i>	<i>what are you doing ?</i>

Table 7: Three sentences which were used as inputs to the VAE, presented with greedy decodes from the mean of the posterior distribution, and from three samples from that distribution.

“ i want to talk to you . ”
“i want to be with you . ”
“i do n’t want to be with you . ”
i do n’t want to be with you .
she did n’t want to be with him .

he was silent for a long moment .
he was silent for a moment .
it was quiet for a moment .
it was dark and cold .
there was a pause .
it was my turn .

Table 8: Paths between pairs of random points in VAE space: Note that intermediate sentences are grammatical, and that topic and syntactic structure are usually locally consistent.

VQ-VAE

- Vector Quantized VAE (VQ-VAE) learns a continuous codebook, but the encoder outputs discrete codes
- Decoder takes a code and generates a sample conditioned on it

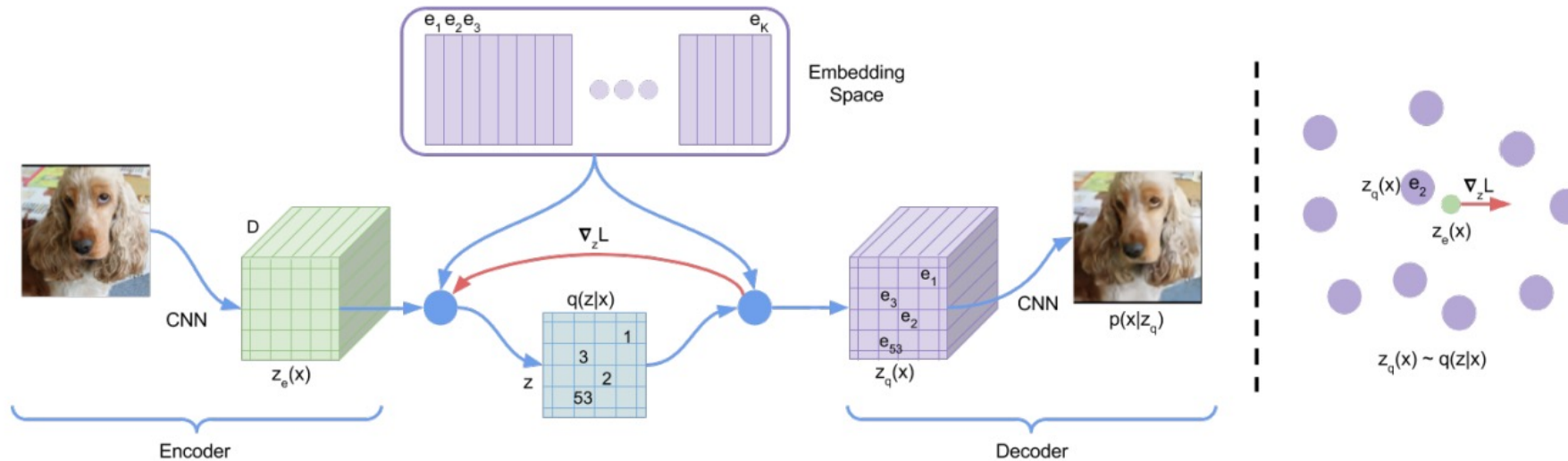
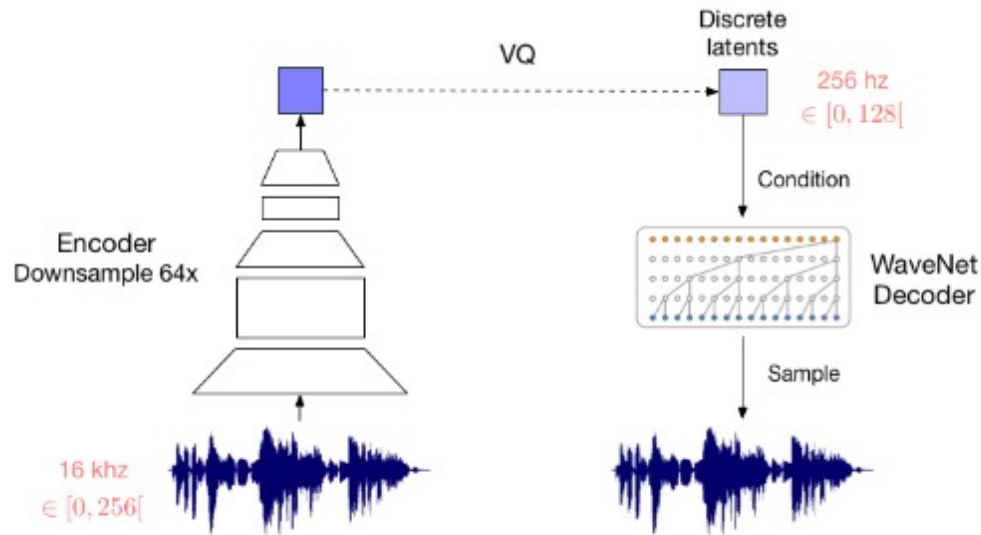


Figure 1: Left: A figure describing the VQ-VAE. Right: Visualisation of the embedding space. The output of the encoder $z(x)$ is mapped to the nearest point e_2 . The gradient $\nabla_z L$ (in red) will push the encoder to change its output, which could alter the configuration in the next forward pass.

VQ-VAE

- Vector Quantized VAE (VQ-VAE) learns a continuous codebook, but the encoder outputs discrete codes
- Decoder takes a code and generates a sample conditioned on it

Example: Generating Audio



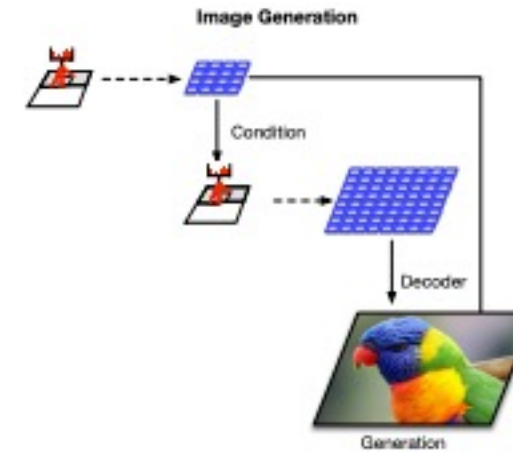
<https://avdnoord.github.io/homepage/vqvae>

VQ-VAE

- VQ-VAE-2 extended the original idea by learning two levels (bottom and top) and a strong prior over the latent space
- Samples from this new model can be convincing even at high-fidelity



(a) Overview of the architecture of our hierarchical VQ-VAE. The encoders and decoders consist of deep neural networks. The input to the model is a 256×256 image that is compressed to quantized latent maps of size 64×64 and 32×32 for the *bottom* and *top* levels, respectively. The decoder reconstructs the image from the two latent maps.



(b) Multi-stage image generation. The top-level PixelCNN prior is conditioned on the class label, the bottom level PixelCNN is conditioned on the class label as well as the first level code. Thanks to the feed-forward decoder, the mapping between latents to pixels is fast. (The example image with a parrot is generated with this model).

- VQ-VAE-2 extended the original idea by learning two levels (bottom and top) and a strong prior over the latent space
- Samples from this new model can be convincing even at high-fidelity



Figure 4: Class conditional random samples. Classes from the top row are: 108 sea anemone, 109 brain coral, 114 slug, 11 goldfinch, 130 flamingo, 141 redshank, 154 Pekinese, 157 papillon, 97 drake, and 28 spotted salamander.

- VQ-VAE-2 extended the original idea by learning two levels (bottom and top) and a strong prior over the latent space
- Samples from this new model can be convincing even at high-fidelity

Figure from Razavi et al.
(2019)



- VQ-VAE-2 extended the original idea by learning two levels (bottom and top) and a strong prior over the latent space
- Samples from this new model can be convincing even at high-fidelity

Figure from Razavi et al.
(2019)



- VQ-VAE-2 extended the original idea by learning two levels (bottom and top) and a strong prior over the latent space
- Samples from this new model can be convincing even at high-fidelity

Figure from Razavi et al.
(2019)



Connections between VAE and Diffusion

Variational Autoencoder

VAE The VAE learns via stochastic gradient variational Bayes (SGVB):

- true posterior: $p_\theta(\mathbf{z} \mid \mathbf{x})$ (intractable)
- variational approx: $q_\phi(\mathbf{z} \mid \mathbf{x})$ (encoder)
- model: $p_\theta(\mathbf{x}, \mathbf{z}) = p_\theta(\mathbf{x} \mid \mathbf{z})p_\phi(\mathbf{z})$
where $p_\theta(\mathbf{x} \mid \mathbf{z})$ (decoder) and $p_\theta(\mathbf{z})$ (Gaussian)
- learning samples an example i :

$$\begin{aligned}\tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)}) &= -D_{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}^{(i)}) \parallel p_\theta(\mathbf{z})) + \log p_\theta(\mathbf{x}^{(i)} \mid \mathbf{z}^{(i)}) \\ [\theta, \phi] &\leftarrow [\theta, \phi] - \nabla_{[\theta, \phi]} \tilde{\mathcal{L}}^B(\theta, \phi; \mathbf{x}^{(i)})\end{aligned}$$

Denoising Diffusion Probabilistic Model

DDPM The DDPM learns in a similar fashion:

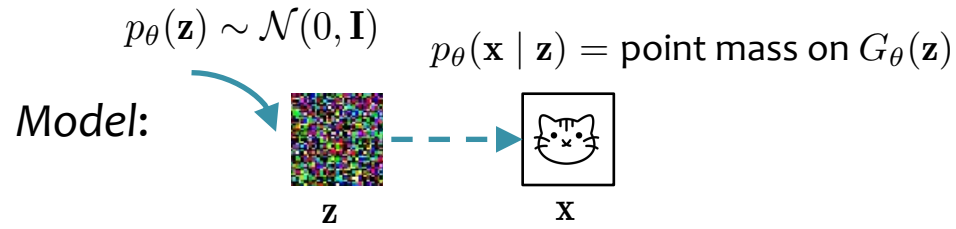
- true posterior: $p_\theta(\mathbf{x}_1, \dots, \mathbf{x}_T \mid \mathbf{x}_0)$ (intractable)
- variational approx: $q_\phi(\mathbf{x}_{0:T})$ (forward process)
- model: $p_\theta(\mathbf{x}_{0:T})$ (reverse process)
- learning only learns θ (i.e. no inference):

$$\begin{aligned}\ell_t(\theta) &\leftarrow \|\epsilon - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \\ \theta &\leftarrow \theta - \nabla_\theta \ell_t(\theta)\end{aligned}$$

GAN → VAE → Diffusion

(in 5 minutes)

GANs

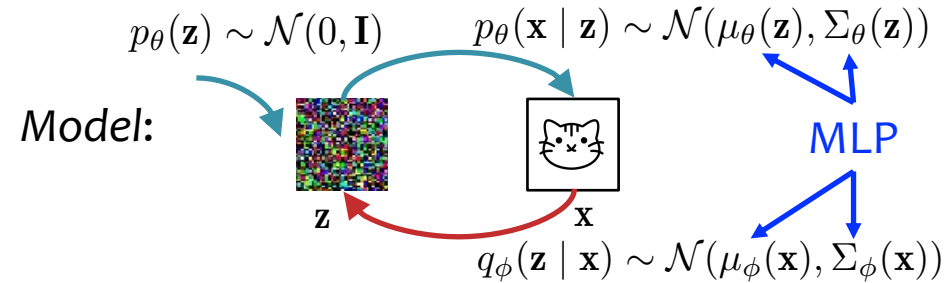


MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: minimax game

VAEs



$$h = \tanh(\mathbf{W}_1 \mathbf{z} + \mathbf{b}_1)$$

$$\mu_{\theta}(\mathbf{z}) = \mathbf{W}_2 h + \mathbf{b}_2$$

$$\Sigma_{\theta}(\mathbf{z}) = (\mathbf{W}_3 h + \mathbf{b}_3) \mathbf{I}$$

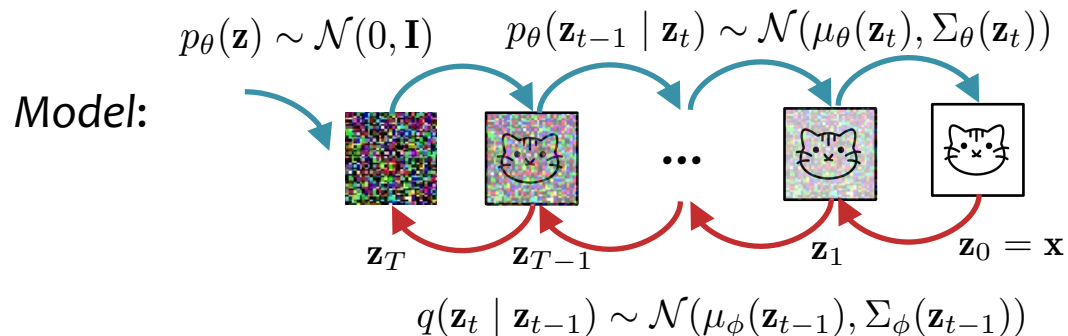
MLE?: $\hat{\theta} = \arg \max_{\theta} \sum_{i=1}^N \log p(\mathbf{x}^{(i)})$ NOPE!

$$p(\mathbf{x}) = \int_{\mathbf{z}} p(\mathbf{x} | \mathbf{z}) p(\mathbf{z}) d\mathbf{z}$$

Learn: $\theta, \phi = \arg \min_{\theta, \phi} \text{KL}(q_{\phi}(\mathbf{z} | \mathbf{x}) || p_{\theta}(\mathbf{z} | \mathbf{x}))$

where $p_{\theta}(\mathbf{z} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}) p(\mathbf{z})$

Diffusion



Learn: $\theta = \arg \min_{\theta} \text{KL}(q(\mathbf{z}_{1:T} | \mathbf{x}) || p_{\theta}(\mathbf{z}_{1:T} | \mathbf{x}))$

where $p_{\theta}(\mathbf{z}_{1:T} | \mathbf{x})$ is true posterior of $p(\mathbf{x} | \mathbf{z}_1) \cdots p(\mathbf{z}_{T-1} | \mathbf{z}_T) p(\mathbf{z}_T)$

MLE?: NOPE!